

ShipNet — Guía de trabajo: Bianca Morales

Rol: Frontend · Sprint 1 (11/09 - 15/09/2026) · Programación Web, UPDS Cochabamba

1. Cómo subir tus cambios al repositorio

El proyecto vive en github.com/angelops1998/shipnet. Nadie sube cambios directo a la rama **main**: cada tarea (issue) se trabaja en su propia rama y se integra mediante un **Pull Request** (PR) que otro integrante revisa antes de aprobar.

1.1 Configuración inicial (solo la primera vez)

1. Acepta la invitación como colaborador que te llegó por correo (o entra a github.com/angelops1998/shipnet/invitations).

2. Clona el repositorio en tu computadora:

```
git clone https://github.com/angelops1998/shipnet.git
cd shipnet
```

3. Crea tu archivo de configuración local a partir de la plantilla. Este archivo **no se sube** a GitHub (está en el `.gitignore`), así que cada uno pone su propia contraseña de MySQL:

```
cp appsettings.Development.json.example appsettings.Development.json
```

Luego abre `appsettings.Development.json` y reemplaza **TU_PASSWORD_AQUI** por la contraseña de tu MySQL local.

4. Verifica que el proyecto compile:

```
dotnet build
```

1.2 Flujo para cada issue

Repite estos pasos por cada issue que tomes. Usa el número de issue (SN-XX) en el nombre de la rama para que sea fácil identificarla.

1. Ponte en **main** y trae lo último que subió el equipo:

```
git checkout main
git pull
```

2. Crea una rama nueva para tu issue (ejemplo con SN-01):

```
git checkout -b feature/sn-01-modelo-er
```

3. Trabaja normalmente. Cuando tengas un avance que valga la pena guardar, revisa qué cambió y haz un commit con un mensaje claro:

```
git status
git add .
git commit -m "SN-01: diagrama ER con entidades y relaciones"
```

Antes de **git add .** mira la lista de **git status**: si aparece `appsettings.Development.json`, `bin/` u `obj/`, algo está mal con el `.gitignore` — avisa antes de commitear.

4. Sube tu rama a GitHub (la primera vez con **-u**; las siguientes basta **git push**):

```
git push -u origin feature/sn-01-modelo-er
```

1.3 Crear el Pull Request

1. Entra a **github.com/angelops1998/shipnet**. Verás un aviso amarillo con tu rama y el botón **Compare & pull request**. Haz clic.
2. **Título:** el mismo del issue, por ejemplo *SN-01: Diseñar el modelo entidad-relación*.
3. **Descripción:** resume qué hiciste y agrega la línea **Closes #N** (donde N es el número del issue en GitHub). Así el issue se cierra solo cuando el PR se aprueba.
4. En **Reviewers** elige a un compañero para que revise. Haz clic en **Create pull request**.
5. Si el revisor pide cambios, corrígelos en la misma rama, haz commit y push de nuevo — el PR se actualiza solo.
6. Cuando esté aprobado, se hace clic en **Merge pull request** y luego **Delete branch**.

1.4 Reglas del equipo

- Nunca hacer **git push** directo a **main**.
- Nunca subir **appsettings.Development.json** (contiene tu contraseña de MySQL).
- Un issue = una rama = un Pull Request.
- Hacer **git pull** en main antes de crear cada rama nueva, para no trabajar sobre código viejo.
- Si al hacer pull aparece un **conflicto**, no lo resuelvas a ciegas: avisa en el grupo.

2. Tu tarea del Sprint 1

Tienes un solo issue este sprint: definir la identidad visual de ShipNet para que en Sprint 2 y 3 todas las pantallas (login, asistencia, evaluaciones) se construyan sobre una base consistente, en vez de que cada vista improvise su estilo. Total: **2 puntos**.

SN-13 — Definir estilo base con Bootstrap 5.3

Sprint	Sprint 1 (11/09 - 15/09)
Puntos	2
Rama sugerida	feature/sn-13-...

Contexto

El proyecto ya incluye Bootstrap en **wwwroot/lib/bootstrap/** y un layout genérico en **Views/Shared/_Layout.cshtml**. Tu trabajo es reemplazar ese look "por defecto" por el de ShipNet. Nunca edites los archivos dentro de **wwwroot/lib/**: todo lo propio va en **wwwroot/css/site.css** y en las vistas Razor.

Pasos tentativos

1. Define la paleta. Elige un color principal para ShipNet (el proyecto es de red local y control de acceso en un laboratorio: azules/verdes tipo "nautico" funcionan bien con el nombre). Bootstrap 5.3 se personaliza sobrescribiendo sus variables CSS en **site.css**:

```
:root {
  --bs-primary: #2b6e71;
  --bs-primary-rgb: 43, 110, 113;
  --bs-body-font-family: "Segoe UI", system-ui, sans-serif;
}

.btn-primary {
  --bs-btn-bg: var(--bs-primary);
  --bs-btn-border-color: var(--bs-primary);
  --bs-btn-hover-bg: #17494c;
  --bs-btn-hover-border-color: #17494c;
}
```

2. Rediseña **Views/Shared/_Layout.cshtml**: navbar con el nombre ShipNet, footer con materia y universidad, y el contenedor central. Deja el navbar preparado para que cambie según el rol (Admin, Docente, Estudiante). Backend conectará la lógica; tú deja la estructura con un condicional Razor para cada bloque de menú:

```
<ul class="navbar-nav me-auto">
  @if (User.IsInRole("Docente"))
  {
    <li class="nav-item"><a class="nav-link" asp-controller="Asistencia" asp-action="Index">Asistencia</a>
    <li class="nav-item"><a class="nav-link" asp-controller="Evaluaciones" asp-action="Index">Evaluaciones</a>
  }
  @if (User.IsInRole("Estudiante"))
  {
    <li class="nav-item"><a class="nav-link" asp-controller="Asistencia" asp-action="Marcar" >Marcar asistencia</a>
  }
  @if (User.IsInRole("Admin"))
  {
    <li class="nav-item"><a class="nav-link" asp-controller="Admin" asp-action="Index">Administración</a>
  }
</ul>
```

Estos enlaces apuntan a controladores que todavía no existen; mientras tanto no rompen nada porque el bloque no se muestra hasta que haya un usuario logueado con ese rol.

3. Define los componentes que todo el equipo va a reutilizar, con estilo consistente:

- **Botones:** primario (acción principal), secundario (cancelar/volver) y peligro (ej. "Cerrar sesión de evaluación").
 - **Cards:** para listar sesiones de asistencia, evaluaciones y equipos.
 - **Alertas:** éxito (asistencia registrada), error (equipo no autorizado), advertencia (tiempo por terminar). Considera un partial `_Alertas.cshtml` que lea `TempData["Mensaje"]` para que backend solo tenga que asignar el texto.
 - **Formularios:** inputs con label, ayuda y estado de validación (clases `is-invalid` / `invalid-feedback` que ya usa ASP.NET con `asp-validation-for`).
 - **Tablas:** estilo para listados (asistencia por grupo, resultados).
4. Convierte **Views/Home/Index.cshtml** en una página de muestra ("guía de estilo") que exhiba cada componente anterior con la paleta aplicada. Sirve como referencia visual para Angel, Baruc y Daniel cuando armen sus vistas.
5. Revisa la página en una ventana angosta (o con las herramientas de desarrollador del navegador en modo móvil): el navbar debe colapsar y las cards apilarse. En el laboratorio se usarán monitores, pero el docente podría abrir el panel desde el celular.
6. Corre **dotnet run**, abre la URL que indica la consola y verifica todo en el navegador antes de abrir el PR.

Criterio de aceptación

- [] `_Layout.cshtml` con navbar y footer de ShipNet, preparado para los 3 roles
- [] `site.css` con la paleta definida sobre variables de Bootstrap
- [] Página de muestra con botones, cards, alertas, formulario y tabla estilizados
- [] El layout no se rompe en pantalla angosta